TP 01: Sécuriser et tester les mots de passe d'ARCADIA

Contexte professionnel

Le premier compte technicien créé sur SRV-AD01 utilisait « Password1 », jugé insuffisant lors d'un contrôle du RSSI. Vous devez corriger la politique et démontrer son efficacité.

Consignes :

1. Configurer la stratégie de mot de passe locale sur “windows server”(longueur ≥ 16, complexité, verrouillage 3 tentatives) et créer un compte « technicien » conforme.
2. Configurer pam_pwquality sur “Debian” avec des exigences équivalentes et l'expiration à 60 jours.
3. Choisir un mot de passe faible et un mot de passe fort ; tester les deux avec cracklib-check et pwscore.
4. Générer des mots de passe robustes avec pwgen

Correction:

cracklib-check et pwscore sont deux outils Linux permettant d'évaluer la qualité d'un mot de passe, mais ils ont des objectifs légèrement différents.

Outil	Fonction	Utilisation
cracklib-check	Vérifie si un mot de passe est faible (dictionnaire, trop simple, etc.)	Test rapide de validité
pwscore	Attribue un score de robustesse au mot de passe selon libpwquality	Évaluation de la force

1. cracklib-check

Installation

sudo apt update

sudo apt install cracklib-runtime

Vérifiez qu'il est installé :

whereis cracklib-check

Utilisation

echo "password" | cracklib-check

Résultat :

password: basé sur un mot du dictionnaire

Autres exemples :

echo "12345678" | cracklib-check

12345678: trop simple/systématique

echo "Admin@202!" | cracklib-check

Admin@2026!: OK

2. pwscore

pwscore fait partie du paquet libpwquality-tools.

Installation

sudo apt update

sudo apt install libpwquality-tools

Vérification :

which pwscore

Utilisation

echo "medkhaled" | pwscore

Résultat : 9

Mot de passe moyen :

echo "medkhaled123" | pwscore

Résultat :53

Mot de passe robuste :

echo "Med!1Srv#27F" | pwscore

Résultat :100

Le score est compris entre 0 et 100 :

Score	Niveau
0–30	Très faible
31–60	Moyen
61–80	Bon
81–100	Très fort

Le score exact dépend de la configuration de libpwquality (longueur minimale, dictionnaire, répétitions, etc.).

3.Générer des mots de passe robustes:

Installation :

sudo apt install pwgen

Générer un mot de passe :

pwgen 20 1

20 = longueur

1 = nombre de mots de passe à générer

Pour forcer des mots de passe plus complexes :

pwgen -sy 20 1

-s : complètement aléatoire

-y : ajoute des symboles

TP 02: Gestionnaire des mots de passe

Retenir un mot de passe long, unique et complexe pour chaque compte est humainement difficile. Un gestionnaire de mots de passe résout ce problème : il génère et stocke des mots de passe robustes dans une base chiffrée, protégée par un seul mot de passe maître.

Présentation

Outil	Fonctionnement	Avantage principal
KeePass	Gestionnaire de mots de passe sécurisé et gratuit. Ce logiciel libre et en français, certifié par l'ANSSI, permet de stocker en sécurité vos mots de passe pour les utiliser dans vos applications. KeePass dispose d'une fonction pour générer des mots de passe complexes aléatoires.	Fonctionne entièrement hors-ligne, données jamais transmises à un tiers
Proton Pass	Gestionnaire de mots de passe open source développé par Proton. Il enregistre vos identifiants dans un coffre-fort chiffréet propose la génération de mots de passe complexes, le remplissage automatique, l'authentification à deux facteurs ainsi que la création d'alias d'adresses e-mail. Une version gratuite est disponible.	
Bitwarden	Coffre synchronisé dans le cloud (ou auto-hébergé), applications et extensions navigateur	Accessible sur tous les appareils, partage sécurisé possible en équipe

KeePass convient bien à un usage local sans dépendance Internet c'est l'outil utilisé dans le TP ci-dessous.

Créer un coffre KeePass pour ARCADIA

Consignes :

1. Installer ou lancer KeePass et créer une nouvelle base de données protégée par un mot de passe maître robuste.
2. Créer des groupes pour organiser les comptes ARCADIA (ex : Serveurs, Applications métier, Comptes techniciens).
3. Ajouter au moins 3 entrées, en utilisant le générateur intégré pour produire un mot de passe conforme

TP 03:Vérification d'intégrité

Nous allons créer un fichier, calculer son empreinte, le modifier, puis observer comment le hachage "détecte" cette modification.

Étape 1 : Créer le fichier original

Ouvrez votre terminal et tapez ces commandes :

```
# Créer un fichier texte avec un contenu simple
echo "Ceci est un document secret important" > document.txt

# Calculer et afficher son empreinte SHA-256
sha256sum document.txt
```

Le terminal vous affichera une longue chaîne de caractères. ***Notez-la ou gardez-la en mémoire.*** *C'est votre "preuve" d'intégrité.*

Étape 2 : Simuler une altération

Maintenant, imaginons qu'une personne malveillante modifie le contenu du fichier sans que vous le sachiez:

```
# Ajouter un simple point à la fin du fichier
echo "." >> document.txt
```

Étape 3 : Vérifier l'intégrité

Relancez la commande de vérification sur le même fichier :

```
sha256sum document.txt
```

Observation

Vous allez constater que l'empreinte affichée est **totalement différente** de celle obtenue à l'étape 1, alors que vous n'avez ajouté qu'un seul caractère (le point).

- **Avant :** 8675... (par exemple)
- **Après :** 5f2d... (totalement différent)

Ce que cela prouve

Dans un contexte réel (téléchargement d'un logiciel ou mise à jour système), si vous aviez comparé le résultat de l'étape 3 avec le hash officiel fourni par l'éditeur, vous auriez vu immédiatement que le fichier a été modifié (ou corrompu). **C'est ainsi que Debian vérifie, par exemple, que les paquets .deb téléchargés depuis les dépôts sont conformes et n'ont pas été altérés en chemin.**

Astuce : Si vous voulez automatiser cela comme les vrais outils système, vous pouvez créer un fichier de contrôle :

1. sha256sum document.txt > document.sha256 (crée le fichier de référence)
2. sha256sum -c document.sha256 (vérifie le fichier par rapport à la référence)

Cela vous affichera un message clair : document.txt: OK ou FAILED.